CloudX Memory Plugin Guide

How the Documentation Archive plugin stores, searches, invalidates, and ports local knowledge

CloudX

2026-06-03

Table of contents

1 Executive Summary

The new memory plugin is implemented as the CloudX `documentation` plugin plus a local FastAPI service called the Documentation Indexer. Its job is to turn uploaded files, allowed local files and directories, URLs, copied text, and transcripts into a portable local knowledge archive that CloudX can search later through plugin hooks, the Documentation tab, HTTP, automation, and voice-aware read paths.

The system is intentionally local-first. The archive lives under `CLOUDX_DOCUMENTATION_DATA_DIR`, defaulting to `.cloudx/documentation`. That directory contains the SQLite catalog, immutable source snapshots, and the Turbovec dense index. Backing up that directory as a unit is the backup story.

The retrieval model is hybrid by default: lexical SQLite FTS5 search and a local hash-based dense vector search are combined with reciprocal rank fusion. Non-active documents remain auditable, but the dense index is rebuilt only from active chunks.

2 Big Picture

The request path has five layers:

1. The browser panel, plugin hooks, HTTP routes, automation, or a Codex skill creates an ingest, search, invalidate, or maintenance request.
2. The CloudX server plugin validates CloudX concerns such as allowed local paths, hook safety, and browser upload limits.
3. `DocumentationClient` forwards the request to the local Documentation Indexer.
4. The indexer extracts content, stores source snapshots, updates SQLite and FTS5, and rebuilds the Turbovec index when active chunks change.
5. Codex tabs receive the documentation system rule and skills automatically through the CloudX overlay path used by built-in rules and skills.

The browser UI and other CloudX surfaces do not directly mutate archive files. They call plugin hooks such as `documentation.search`, `documentation.ingest.path`, and `documentation.invalidate`. The server-side plugin resolves allowed local paths and forwards requests through `DocumentationClient` to the local indexer.

The indexer owns all persistence. It initializes the archive directories, extracts text, writes source snapshots, stores document and chunk metadata in SQLite, updates FTS5 rows through SQLite triggers, and rebuilds the Turbovec file after ingest or invalidation.

3 Components

3.1 CloudX Plugin

Source: `apps/server/src/plugins/DocumentationPlugin.ts`

Registers the `documentation` plugin, exposes hooks, contributes the UI renderer, enforces path policy for local path ingest, and declares default documentation rules and skills as automatic CloudX system contributions.

3.2 HTTP Client

Source: `apps/server/src/documentation/DocumentationClient.ts`

Wraps the indexer REST API, applies request timeout, bounds response size, and normalizes service errors.

3.3 Web Panel

Source: `apps/web/src/ui/DocumentationPanel.tsx`

Provides search, filters, ingest forms, invalidation buttons, manifest view, and index rebuild controls.

3.4 Indexer API

Source: `services/documentation-indexer/src/cloudx_documentation_indexer/main.py`

Defines FastAPI routes for health, stats, documents, ingest, search, invalidate, delete, and rebuild.

3.5 Archive Engine

Source: `services/documentation-indexer/src/cloudx_documentation_indexer/archive.py`

Stores snapshots and metadata, extracts text, chunks content, embeds chunks, builds the Turbovec index, searches, and records invalidation history.

4 Archive Layout

By default, the archive root is `.cloudx/documentation`. It can be changed with `CLOUDX_DOCUMENTATION_DATA_DIR`.

The archive root contains:

Path	Purpose
`catalog.sqlite`	SQLite catalog for documents, chunks, FTS5 search rows, and invalidation events.

Path	Purpose
`snapshots/<sha256>/...`	Immutable source bytes captured at ingest time. URL snapshots may also include `metadata.json` with final URL, content type, ETag, and last-modified headers. Rich extraction sidecars live under `snapshots/<sha256>/extracted/`.
`indexes/local-hash-64/chunks.tvim`	Turbovec dense index built from active chunks only.
`indexes/local-hash-64/manifest.json`	Rebuild metadata for schema version, embedding profile, Turbovec version, index format, dense threshold, and active chunk count.

The portable manifest endpoint walks the archive root and returns each file path, byte size, and SHA-256. The guide-level rule is simple: stop writes and back up the whole archive directory, not just the SQLite database or the vector file.

5 Ingestion Flow

The plugin supports four ingest paths.

5.1 Browser Upload Ingest

The Documentation panel defaults to upload mode. The browser sends file bytes to `POST /api/documentation/upload`; the CloudX server enforces the configured upload cap and forwards the bytes to the indexer's multipart `POST /ingest/upload` route.

Upload ingest stores the original filename and content type in snapshot metadata, infers a source type when the UI is left on `auto`, and records a stable `upload://<safe-name>` URI. This path is the preferred user workflow for datasheets, PDFs, images, Markdown, copied exports, and other files that are on the user's workstation but not necessarily visible as a local path to the indexer process.

5.2 Local Path Ingest

`documentation.ingest.path` accepts a file or directory. Before the request reaches the indexer, CloudX resolves the path through `PathPolicy`, so the file must be under configured allowed roots. If a directory is passed, the indexer recursively ingests supported documentation file suffixes.

Supported suffixes include Markdown, text, PDF, HTML, JSON, YAML, XML, CSV, SRT/VTT, Python, TypeScript, JavaScript, C/C++, Rust, CSS, AsciiDoc, LaTeX, and images (`.png`, `.jpg`, `.jpeg`, `.tif`, `.tiff`, `.bmp`, `.webp`).

The supported source-type labels are `datasheet`, `book`, `website`, `repo_code`, `readme`, `media`, `image`, and `text`. Source type describes the source for filtering and skill behavior; extraction is selected from the actual file suffix, content type, and file signature. That means setting `sourceType` to `datasheet` no longer forces a Markdown, text, or image file through the PDF extractor.

Input class	Examples	Extraction behavior
PDF	`.pdf`, `application/pdf`, `%PDF-` bytes	Page text, tables, and rendered visual page artifacts.
Image	`.png`, `.jpg`, `.jpeg`, `.tif`, `.tiff`, `.bmp`, `.webp`, `image/*`	Normalized PNG artifact plus format, size, mode, and frame metadata.
HTML	`.html`, `.htm`, `text/html`	Text after removing script, style, template, and noscript content.
Text and code	`.md`, `.txt`, `.json`, `.yaml`, `.xml`, `.csv`, `.py`, `.ts`, `.tsx`, `.js`, `.c`, `.cpp`, `.h`, `.rs`, `.css`, `.adoc`, `.tex`	UTF-8 text decode with replacement for invalid bytes.
Captions and transcripts	`.srt`, `.vtt`, manually supplied transcript text	Text chunks marked as media when requested.

5.3 URL Ingest

`documentation.ingest.url` downloads a URL with redirects enabled and a 20 second timeout. The indexer records the original URL, final URL, content type, ETag, and last-modified header when those values are available.

If the caller provides a transcript, the URL is ingested as media text without downloading page content. If the source type is `media` or the URL is recognized as YouTube, the indexer tries to fetch the transcript and stores it as media text.

5.4 Text Ingest

`documentation.ingest.text` stores direct text, copied source material, or manually supplied transcripts. It requires a title, URI, source type, and non-empty text. If the text is blank after trimming, the indexer rejects the request.

5.5 Extraction And Chunking

PDF input is extracted with a table-aware pipeline. Page text uses `page N` locators; detected tables become searchable Markdown chunks and portable sidecars under `extracted/tables/`; pages with visual objects are rendered to PNG sidecars under `extracted/figures/` so graphs, plots, flowcharts, schematics, and layout-heavy pages can be inspected later. Image input is normalized to a PNG sidecar under `extracted/images/` and indexed with format, size, mode, frame count, and visual-artifact metadata. HTML input is parsed with script, style, template, and noscript content removed. Other input is decoded as text. Extracted spans are split into paragraph-like chunks with a maximum target size of 1200 characters.

This is similar to the datasheet-analysis workflow in the important ways for storage and recall: tables become standalone artifacts, visual pages are preserved as rendered images, and each chunk carries a locator. It is not OCR. Text inside a graph or scanned page is searchable only when it is present in the PDF text layer or table extraction output; otherwise the graph or flowchart is preserved as an artifact for inspection and citation.

Each document ID is deterministic for a `(uri, content_sha256)` pair. Re-ingesting the same URI and content replaces its chunks and keeps the document active.

6 Storage Model

SQLite has three durable tables and one FTS5 virtual table:

Store	Meaning
`documents`	One row per ingested source, including title, source type, URI, snapshot path, content hash, state, collection, tags, and timestamps.
`chunks`	Extracted text chunks with locators and state. Each chunk belongs to a document.
`chunks_fts`	SQLite FTS5 table over chunk text and locator. Triggers keep it synchronized on chunk insert, update, and delete.
`invalidation_events`	Append-only audit trail for state transitions, with previous state, next state, reason, and timestamp.

The dense index is separate from SQLite. It is a Turbovec `IdMapIndex` using:

Setting	Value
Embedding profile	`local-hash-64`
Embedding dimension	`64`
Turbovec bit width	`4`
Index format	`tvim`
Dense-only hybrid threshold	`0.2`

The embedding is local and deterministic. The indexer tokenizes lower-case terms, hashes each token, updates a 64-dimensional signed vector, and normalizes it. There is no remote embedding model involved in this implementation.

7 Search Flow

The search API accepts:

Field	Behavior
`query`	Required, non-empty search text.
`limit`	Must be between 1 and 100.
`states`	Defaults to `active`.
`sourceTypes`	Optional filter such as `datasheet`, `book`, `website`, `repo_code`, `readme`, `media`, `image`, or `text`.
`collection`	Optional exact collection filter.

Field	Behavior
`mode`	`hybrid`, `dense`, or `lexical`; defaults to `hybrid`.

`lexical` mode uses SQLite FTS5 and BM25 ordering. It also boosts strict term matches and identifier-like terms such as register names, part numbers, and planted validation IDs so exact source facts are not buried by dense neighbors. `dense` mode searches Turbovec with an allowlist of eligible chunk IDs. `hybrid` mode runs both, drops weak dense-only candidates below `0.2`, and combines the ranked lists with reciprocal rank fusion plus lexical relevance.

The result payload includes the chunk ID, document ID, title, source type, URI, locator, snippet, fused score, optional dense and lexical scores, and citation metadata with the content SHA and snapshot path. That makes search results usable as source-grounded references instead of free-floating text.

8 State And Invalidation

New and re-ingested documents become `active`. The non-active states are:

State	Intended meaning
`stale`	The source is outdated.
`superseded`	A newer source replaced this one.
`revoked`	The source is wrong or should no longer be trusted.
`quarantined`	The source has trust, provenance, or extraction concerns.
`deleted`	The source was removed from normal active use.

Invalidation updates both the document row and its chunks, records an invalidation event, then rebuilds the Turbovec index. Removal is implemented as invalidation to `deleted`. The rows remain available for audit and explicit non-active searches.

9 API And Hook Map

9.1 Read Hooks

- `documentation.health`: `GET /health`. Returns health, archive root, schema, embedding profile, index path, and portability flag.
- `documentation.stats`: `GET /stats`. Returns document and chunk counts plus portable paths.
- `documentation.portableManifest`: `GET /portable-manifest`. Returns the complete archive file manifest.
- `documentation.documents.list`: `GET /documents`. Lists documents and defaults to active state.
- `documentation.documents.get`: `GET /documents/{id}`. Fetches one document with chunks and invalidation history.
- `documentation.search`: `POST /search`. Searches the archive and is exposed to plugin, UI, HTTP, automation, and voice.

9.2 Write And External Hooks

- `documentation.ingest.path`: `POST /ingest/path`, safety `write`. Ingests allowed local files or directories.
- `documentation.ingest.url`: `POST /ingest/url`, safety `external`. Downloads or transcript-ingests external content.
- `documentation.ingest.text`: `POST /ingest/text`, safety `write`. Ingests copied text or manually supplied transcript text.
- `documentation.invalidate`: `POST /invalidate`, safety `write`. Marks a document non-active with a reason.
- `documentation.remove`: `DELETE /documents/{id}`, safety `write`. Marks a document deleted.
- `documentation.rebuildIndex`: `POST /rebuild-index`, safety `write`. Rebuilds Turbovec from active SQLite chunks.

The `DocumentationClient` defaults to `http://127.0.0.1:7820`, a 30 second timeout, and an 8 MiB maximum response body. Failed service responses are converted into plain error messages when the response includes `detail`, `message`, or `error`.

Binary upload is exposed through HTTP rather than a plugin hook because hook inputs are JSON-shaped. The browser route is `POST /api/documentation/upload` with an `application/octet-stream` body and metadata in query parameters. The indexer route is `POST /ingest/upload` with multipart form data.

10 UI Workflow

In CloudX, create a Documentation tab. The panel gives a compact operational view:

1. Refresh archive stats and active document list.
2. Search with `hybrid`, `dense`, or `lexical` mode.
3. Filter by source type, state, and collection.
4. Add knowledge from an uploaded file, path, URL, or text.
5. Leave source type on `auto` for uploads, paths, and URLs unless the user knows the source category.
6. Supply optional media transcripts when URL ingest is set to source type `media`.
7. Mark search results `stale`, `revoked`, `superseded`, or `quarantined`.
8. Remove a document from active search.
9. Load the portable manifest and rebuild the Turbovec index.

The UI defaults search state to `active`, ingest mode to upload, source type to `auto`, and search mode to `hybrid`.

11 Default Rule And Skills

The plugin contributes this universal CloudX system rule automatically at server startup:

Rule	Purpose
`documentation-ingest-evidence`	Download or otherwise capture task evidence such as datasheets, sample code, vendor documentation, screenshots, flowcharts, API references, and local notes into the documentation knowledge base before relying on it; preserve precise metadata and invalidate older conflicting records.

The plugin contributes four CloudX system skills automatically at server startup:

Skill	Purpose
`documentation-search`	Search the local archive and use active source-grounded results.
`documentation-ingest`	Add local files, directories, websites, copied text, or transcripts.
`documentation-invalidate`	Find and invalidate stale, wrong, superseded, quarantined, or deleted sources.
`documentation-archive-control`	Inspect health, stats, portable manifest, and rebuild status.

Each skill tells Codex to read `CLOUDX_DOCUMENTATION_URL` first. When Codex tabs are launched from CloudX, the server exports that URL to child processes.

The injection path is generic. Plugins expose `ruleContributions` and `skillContributions`, `syncPluginContributions` writes them into the CloudX `system-rules/` and `system-skills/` catalogs, and each Codex home overlay materializes all system rules into `AGENTS.override.md` plus all system skills under `skills/cloudx-system/`. A future plugin should follow the same pattern: use IDs prefixed with the plugin ID, provide a single-line rule text or complete `SKILL.md` body through the contribution, and let the catalog and overlay handle availability for every Codex tab.

12 Setup And Operations

Install and start the indexer:

```
npm run documentation:setup
npm run documentation:start
```

The equivalent explicit start command is:

```
CLOUDX_DOCUMENTATION_DATA_DIR=.cloudx/documentation \
services/documentation-indexer/.venv/bin/cloudx-documentation-indexer \
  --host 127.0.0.1 --port 7820
```

CloudX defaults to:

```
CLOUDX_DOCUMENTATION_URL=http://127.0.0.1:7820
CLOUDX_DOCUMENTATION_DATA_DIR=.cloudx/documentation
```

Manual backup:

```
tar -czf cloudx-documentation-$(date +%F).tar.gz -C .cloudx documentation
```

Manual restore:

```
mkdir -p .cloudx
tar -xzf cloudx-documentation-YYYY-MM-DD.tar.gz -C .cloudx
npm run documentation:start
curl -sS -X POST http://127.0.0.1:7820/rebuild-index
```

Run rebuild after restore when active document states changed, the service version changed, or you want to prove the Turbovec file can be reconstructed from SQLite chunks.

13 Rendering This PDF

The Ubuntu installer installs the render toolchain used for this guide: Quarto `1.9.38`, Pandoc, and TeX Live's XeLaTeX/LuaLaTeX engines. Regenerate the PDF from the Quarto source with:

```
npm run docs:memory:pdf
```

That command reads `docs/MEMORY_PLUGIN_GUIDE.qmd` and writes `docs/MEMORY_PLUGIN_GUIDE.pdf`.

14 What Is Tested

The indexer tests cover:

- Ingesting uploaded files, PDFs, local directories, image files, HTML websites, README/code-like files, and media transcripts.
- Extracting PDF table sidecars, rendered visual artifacts, and normalized image artifacts.
- Searching with hybrid retrieval, lexical-only behavior, strict identifier recall, and dense-only fallback above the configured threshold.
- Invalidation, deletion, portable manifest generation, archive reopen/restore behavior, FastAPI controls, and CLI help.

The server plugin tests cover hook registration, safety classes, local path policy enforcement, browser upload forwarding, automatic plugin system-rule and system-skill contributions, generic plugin contribution validation, and Codex overlay injection. The web panel tests cover upload ingest, visible text ingest, and search hook calls. The client tests cover JSON POST behavior, multipart upload behavior, browser upload request construction, and propagation of service error messages.

A realistic validation runner in `debug_tooling/documentation-validation/run_validation.py` downloads or reuses public Wikipedia pages, the TurboQuant arXiv PDF, GMSL2 datasheet PDFs, and

a generated 2500-record mock corpus with planted facts. The latest required validation run ingested 6 documents into 3347 active chunks, produced 1866 table CSV files, 1866 table Markdown files, and 469 rendered figure PNGs, then passed required hybrid, lexical, rebuild, and stale-state checks. Dense-only checks are retained as diagnostics because the current dense profile is a deterministic local hash embedding, not a semantic embedding model.

15 Current Boundaries

- The dense embedding model is deterministic local hashing, not a semantic model from a remote embedding service.
- Dense-only mode is diagnostic for this implementation. Use hybrid as the default user-facing recall mode.
- Dense search is restricted to active chunks because only active chunks are loaded into Turbovec.
- URL ingest supports only HTTP and HTTPS, performs direct fetches or transcript ingest, and does not crawl a site recursively.
- Browser uploads and URL downloads are capped at 256 MiB.
- PDF graph and flowchart extraction preserves rendered visual artifacts, but it does not OCR labels or infer graph semantics.
- Path ingest is limited by CloudX path policy and the supported file suffix list.
- Backup is directory-level. Copying only `catalog.sqlite` or only `chunks.tvim` is incomplete.

16 Source Inspection Commands

These are the commands used to ground this guide in the repository:

```
rg --files | rg '(^README.md$|^docs/SETUP.md$|documentation|Documentation)'
rg -n "DocumentationArchive|TURBOVEC|ingest_|search|invalidate" \
  services/documentation-indexer/src/cloudx_documentation_indexer/archive.py
rg -n "CLOUDX_DOCUMENTATION|/ingest|/search|/invalidate|/rebuild-index" \
  services/documentation-indexer/src/cloudx_documentation_indexer/main.py
rg -n "documentation\.|defaultDocumentationSkills|PathPolicy" \
  apps/server/src/plugins/DocumentationPlugin.ts \
  apps/server/src/documentation/DocumentationClient.ts
rg -n "Documentation|Manifest|Rebuild|Add Knowledge|CLOUDX_DOCUMENTATION" \
  README.md docs/SETUP.md apps/web/src/ui/DocumentationPanel.tsx package.json
rg -n "def test_|portable|rebuild|dense|hybrid|FastAPI" \
  services/documentation-indexer/tests/test_archive.py
rg -n "DocumentationPlugin|documentation\.search|automationSafety|callHook" \
  apps/server/src/plugins/DocumentationPlugin.test.ts \
  apps/web/src/ui/DocumentationPanel.test.ts \
  apps/server/src/documentation/DocumentationClient.test.ts
PYTHONPATH=services/documentation-indexer/src \
  services/documentation-indexer/.venv/bin/python \
  debug_tooling/documentation-validation/run_validation.py \
  --root /tmp/cloudx-documentation-validation --mock-count 2500 --skip-download
npm run docs:memory:pdf
```